



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G2

System Integration and Architecture

System Design Document (SDD)

Project Title: SariTrack

Prepared By: Tapales, Christian Kyle Bayarcal

Version: 1.1

Date: May 8, 2026

Status: Final

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
1.0	February 23, 2026	Christian Kyle Tapales	Baseline version for development	Approved
1.1	May 8, 2026	Christian Kyle Tapales	Updated to reflect as-built tech stack (JS/XML) and final features.	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

SariTrack is a multi-tenant SaaS (Software as a Service) platform designed to modernize the traditional sari-sari store ecosystem. The system provides a digital point-of-sale (POS) experience, enabling store owners to move away from manual paper-based logging. It features a **Spring Boot** backend, a **React** web dashboard for management, and an **Android** mobile application for real-time sales scanning.

1.2 Objectives

1. Develop a fully functional multi-tenant sari-sari store management system featuring secure JWT authentication, Google OAuth, and Role-Based Access Control (RBAC) for Software Admins and Store Vendors.
2. Implement a three-tier architecture using Spring Boot for the backend, React for the web management portal, and Android with XML-based layouts for the mobile POS application.
3. Create RESTful APIs for communication between all system components, integrating external services including PayMongo Sandbox for debt settlement, Currency Exchange APIs for real-time rates, and SMTP for automated email notifications.
4. Design a responsive and intuitive user interface optimized for desktop-based inventory management and mobile-based barcode scanning to facilitate a digital cashier experience.
5. Ensure strict data isolation between vendors, secure file storage for product images, and adherence to professional software architecture patterns.

1.3 Scope

Included Features:

- User registration and **JWT-based authentication**.
- Product catalog management with search and **barcode scanning** capabilities.
- Product listing with search functionality
- Shopping cart management (add, remove, update quantities) to facilitate multi-item transactions.
- Checkout process with customer information collection and digital credit settlement via PayMongo Sandbox.

- Admin panel/dashboard for global vendor , sales and product monitoring.
- Responsive web interface for centralized business management and financial analytics.
- Native Android mobile application developed using Kotlin and XML-based layouts .
- PostgreSQL database with a normalized relational schema and SMTP-integrated notification system.

Excluded Features:

- **Real payment gateway integration :** The system uses a Sandbox environment only, no actual financial assets are transferred.
- **Automated Inventory Reordering:** The system tracks stock but does not automatically place orders with external suppliers.
- **Product Reviews and Ratings:** The platform is a utility tracker for owners/staff, not a public-facing marketplace for consumers.
- **Native Push Notifications:** Real-time updates are handled via polling or manual refresh rather than OS-level push services.
- **Multi-Store Inventory Sharing:** Inventory is strictly isolated per Vendor; vendors cannot view or share stock with other vendors .

1.0 INTRODUCTION

1.1 Purpose

This document serves as the formal design specification for **SariTrack**, a multi-vendor business management system. It provides the architectural blueprint, API contracts, and database schema necessary to guide the development of a unified ecosystem consisting of a Spring Boot backend, a React web portal, and a Kotlin/XML mobile application.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: SariTrack

Domain: SaaS / Retail Management

Primary Users:

1. **Software Owner (Admin):** Manages Vendor accounts and platform health.
2. **Store Owner (Vendor):** Manages store inventory, sales, and customer credits.

Problem Statement: Sari-sari store owners rely on fragile paper-based "listahan" systems that are prone to calculation errors, lack data backups, and offer no way to accept digital debt settlements.

Solution: A digital POS and credit tracker that provides secure data isolation, integrated currency conversion for remittance-based pricing, and sandbox digital payment options.

2.2 Core User Journeys

Journey 1: Vendor Onboarding and Store Setup

1. Vendor visits the web application and clicks "Login with Google."
2. System authenticates via **Google OAuth 2.0** and creates a unique Vendor profile.
3. Vendor navigates to "Inventory" and uploads product images and details.
4. System confirms the "Welcome" email was sent via **SMTP**.

Journey 2: The Point of Sale (Mobile)

1. Vendor opens the mobile app and authenticates via **JWT**.
2. Vendor uses the phone camera to scan a product barcode.
3. System retrieves product details and checks current stock.
4. Vendor adds items to the order and clicks "Checkout."
5. System deducts stock and records the **Order > OrderItems** transaction.

Journey 3: Digital Credit Settlement

1. Vendor identifies a customer with an outstanding balance in the "Listahan" module.
2. Vendor selects "Settle via Digital Payment."
3. The system initiates a **PayMongo Sandbox** checkout session.

4. Upon successful test payment, the system updates the ledger and sends an email receipt.

2.3 Feature List (MoSCoW)

MUST HAVE

- **Security:** JWT Auth, Google OAuth, BCrypt hashing, and `/me` endpoint.
- **Core Module:** Product CRUD and Order > OrderItems lifecycle.
- **Integrations:** File Upload (Product Images), External Currency API, and SMTP Email.
- **Roles:** Software Admin (platform level) vs. Vendor (store level).
- **Payment:** PayMongo Sandbox for debt settlement.

SHOULD HAVE

- Daily sales analytics, low-stock indicators, customer credit history, Professional PDF/CSV Exports.
- Low-stock visual indicators on the dashboard.
- Customer credit history logs.

COULD HAVE

- Real-time stock alerts via WebSockets.
- Barcode generation for non-labeled items.

WON'T HAVE

- Real-money production payments.
- Push notifications.
- Automated supplier procurement.

2.4 Detailed Feature Specifications

Feature: Authentication & Identity

- **Screens:** Login, Registration, Profile (`/me`)
- **Methods:** Standard Email/Password and **Google OAuth 2.0**.
- **Security:** JWT storage in the mobile app and hashed passwords in the DB.
- **API Endpoints:** POST `/api/auth/login`, POST `/api/auth/google`, GET `/api/users/me`

Feature: Inventory Management (Core CRUD)

- **Screens:** Product List, Add/Edit Product.

- **Integrations:** **File Upload** for product images and **Barcode Scanning** via mobile.
- **Validation:** Barcode uniqueness, positive pricing, and non-negative stock.
- **API Endpoints:** GET /api/products, POST /api/products (Admin/Vendor only), PUT /api/products/image

Feature: Shopping Cart

- **Screens:** Cart View
- **Functions:** Add product, remove product, update quantity
- **Persistence:** Database storage per user
- **API Endpoints:** GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}

Feature: Digital POS & Order Module

- **Logic:** One-to-Many relationship (One Order contains multiple OrderItems).
- **Calculation:** Automated total and sub-total calculation in the backend service layer.
- **API Endpoints:** POST /api/orders (Atomic transaction: creates order and updates stock).

Feature: Digital Settlement (Payment Integration)

- **Logic:** External payment gateway processing in Sandbox mode.
- **Integration:** **PayMongo API** for session creation and success/failure handling.
- **API Endpoints:** POST /api/payments/create-session, POST /api/payments/webhook

2.5 Acceptance Criteria

AC-1: Multi-Tenant Role Restriction

- **Given** I am logged in as a **Vendor**.
- **When** I attempt to access the "Manage All Vendors" endpoint.
- **Then** the system should return a **403 Forbidden** status code.

AC-2: Atomic Stock Deduction

- **Given** a product has a stock of 10.
- **When** a Vendor completes a sale of 2 units.
- **Then** the database must record the order AND the stock must be exactly 8.
- **And** if the order fails, the stock must remain at 10.

AC-3: External Integration (Currency & Email)

- **Given** a Vendor completes a digital debt settlement.
- **When** the PayMongo Sandbox returns a "Success" status.

- **Then** the system must display the live PHP/USD exchange rate from the External API.
- **And** the system must send an automated email receipt to the store owner.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: ≤ 2 seconds for 95% of requests to ensure the barcode scanner does not lag during peak store hours.
- Database Efficiency: Complex queries, especially those involving the Order > OrderItems relationship, must complete within 500ms through proper indexing and normalization.
- Concurrency: The system must support at least 50 concurrent vendors across different stores without any cross-tenant data leakage or performance degradation.
- Mobile App Cold Start: The Android application must be fully interactive within 3 seconds to facilitate rapid sales logging.
- Throughput: The backend must handle a minimum of 50 transactions per second (TPS) during peak periods.

3.2 Security Requirements

- **Identity Management:** Mandatory **JWT (JSON Web Token)** authentication for all protected routes, with tokens stored securely in the Android EncryptedSharedPreferences and React localStorage.
- **Data Isolation (SaaS Security):** Every database query must be scoped by a vendor_id to ensure that one store owner cannot access the inventory or sales data of another.
- **Password Security:** All passwords must be hashed using **BCrypt**. Plain-text passwords must never be stored.
- **Role-Based Access Control (RBAC):** API-level verification using Spring Security to ensure only the **Software Admin** can manage vendor accounts and only **Vendors** can manage store-specific CRUD operations.
- **Integrations Security:** * **Google OAuth 2.0** for secure, third-party authenticated onboarding.
 - **HTTPS/TLS** encryption for all data in transit between the clients and the Spring Boot API.
- **Input Protection:** Mandatory validation against SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

3.3 Compatibility Requirements

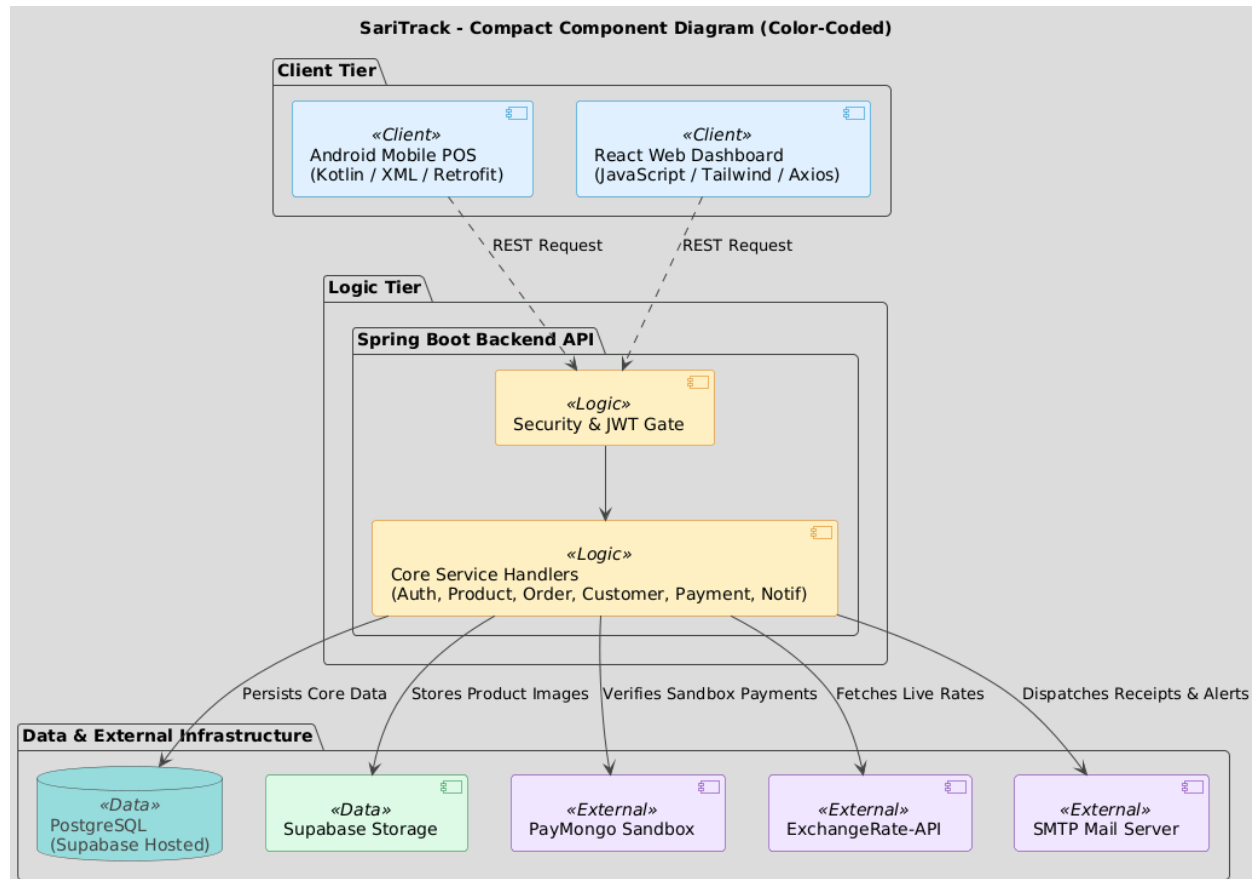
- **Mobile Application:** Strictly compatible with **Android API Level 34 (Android 14)** as per the IT342 final implementation requirements.
- **Web Browsers:** Optimized for the latest two versions of Google Chrome, Mozilla Firefox, Safari, and Microsoft Edge.
- **Screen Responsiveness:**
 - **Mobile (360px+):** Optimized for portrait barcode scanning and quick-checkout XML layouts.
 - **Desktop (1024px+):** Optimized for the Vendor Dashboard, Product CRUD forms, and detailed analytics charts.
- **Backend Architecture:** Developed using **Spring Boot 3.x** to ensure compatibility with modern cloud deployment environments like Railway or Render.

3.4 Usability Requirements

- **Operational Speed:** A trained vendor should be able to complete a 5-item barcode checkout process within **30 seconds**.
- **Navigation:** Consistent sidebar/navigation drawers across both the React web portal and the Kotlin mobile app to reduce the learning curve for store owners.
- **Error Recovery:** Provide clear, user-friendly error messages (e.g., "Product Not Found" or "Insufficient Stock") with actionable recovery options instead of raw stack traces.
- **Accessibility:** Adherence to **WCAG 2.1 Level AA** standards for web components to ensure the system is usable by all store owners.
- **Mobile Targets:** All interactive elements in the Android XML layouts must have a minimum touch target size of **48x48dp** for ease of use in fast-paced retail environments.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA.
- **Database:** PostgreSQL (Hosted on Supabase).
- **Web Frontend:** React 18, JavaScript (ES6+), Tailwind CSS 4, Axios.
- **Mobile:** Kotlin, XML Layouts (Classic View System), Retrofit 2, Room DB.
- **External Services:** Supabase Storage, PayMongo API, ExchangeRate API.

Note on Architectural Completeness: Due to the complexity of multi-tenant data isolation and localized client-side caching implementations, the diagram above highlights high-level routing boundaries. For the fully extended, high-resolution component blueprint detailing granular package configurations please access our production asset board:

<https://drive.google.com/file/d/17KLiL9qTrtRiktWO3hYt3KcyFyajlHJA/view?usp=sharing>

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]/api
Format	application/json
Authentication	Bearer Token (JWT)
Standard Success Envelope Structure	<pre>{ "success": true, "data": { ... }, "message": "Operation executed successfully", "details": null, "timestamp": "2026-05-24T15:08:00Z" }</pre>
Standard Error Envelope Structure	<pre>{ "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed" }, "message": "Method argument validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" }, "timestamp": "2026-05-24T15:08:00Z" }</pre>

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses

Authentication	None
Request Payload	<pre>{ "email": "vendor@email.com", "password": "SecurePassword123", "firstName": "Kyle", "lastName": "Tapales" }</pre>
Response Structure	<pre>{ "user": { "email": "vendor@email.com", "firstName": "Kyle", "lastName": "Tapales", "role": "VENDOR" }, "accessToken": "eyJhbGciOi...", "refreshToken": "eyJhbGciOi..." }</pre>

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "vendor@email.com", "password": "SecurePassword123" }</pre>
Response Structure	<pre>{ "user": { "email": "vendor@email.com", "firstName": "Kyle", "lastName": "Tapales", "role": "VENDOR" }, "accessToken": "eyJhbGciOi...", "refreshToken": "eyJhbGciOi..." }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request

- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

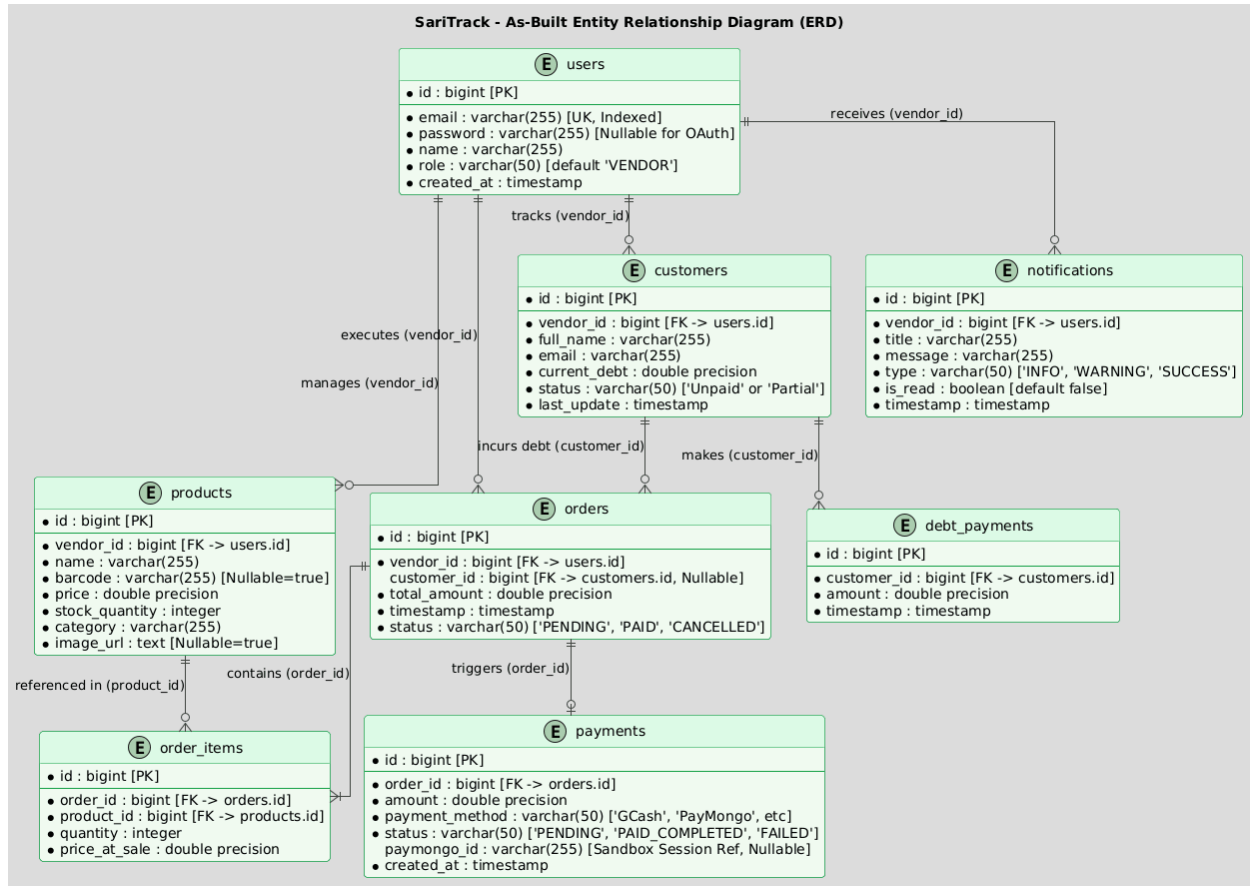
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found

- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many:** users → products (A Store Vendor manages multiple products isolated by their vendor context).
- **One-to-Many:** users → customers (A Store Vendor tracks multiple customers in their decentralized credit ledger).
- **One-to-Many:** users → orders (A Store Vendor executes multiple store transactions).

- **One-to-Many:** users → notifications (A Store Vendor receives multiple system activity alerts and stock warnings).
- **One-to-Many:** customers → orders (A customer optionally incurs outstanding debt across multiple credit transactions).
- **One-to-Many:** customers → debt_payments (A customer makes multiple physical cash or digital sandbox payments to reduce their ledger debt balance).
- **One-to-Many:** orders → order_items (An atomic transaction contains one or multiple individual product rows).
- **Many-to-One:** order_items → products (Individual transaction item entries reference catalog product definitions).
- **One-to-One:** orders → payments (A finalized transaction optionally triggers a corresponding digital sandboxed checkout or local ledger log).

Key Tables:

1. **users** - Stores hashed vendor or admin profile accounts, authentication credentials, and platform security access roles.
2. **products** - Manages the multi-tenant product catalog inventory, mapped categories, images, and scanning barcode attributes.
3. **customers** - Operates the structural "Listahan" tracking table, defining credit caps, contact paths, and standing balances.
4. **orders** - Captures atomic sales receipts, tracking whether an item is settled in cash or logged under a customer debt ledger.
5. **order_items** - Maps multi-item transactional arrays, preserving snapshot sale prices for historical ledger consistency.
6. **payments** - Tracks checkout logs, processing transaction types alongside automated PayMongo sandbox webhook response tokens.
7. **debt_payments** - Records credit ledger deductions whenever a debtor partially or completely settles their outstanding store liabilities.
8. **notifications** - Drives the real-time alerting layer, warning vendors of critical events or low-stock catalog variations.

Table Structure Summary:

- **users:** id (PK), email (UK), password, name, role (ADMIN/VENDOR), created_at
- **products:** id (PK), vendor_id (FK), name, barcode, price, stock_quantity, category, image_url
- **customers:** id (PK), vendor_id (FK), full_name, email, current_debt, status (Unpaid/Partial), last_update
- **orders:** id (PK), vendor_id (FK), customer_id (FK, Nullable), total_amount, timestamp, status (PENDING/PAID/CANCELLED)
- **order_items:** id (PK), order_id (FK), product_id (FK), quantity, price_at_sale
- **payments:** id (PK), order_id (FK), amount, payment_method (GCash/PayMongo/Cash), status (PENDING/PAID_COMPLETED/FAILED), paymongo_id (Nullable), created_at
- **debt_payments:** id (PK), customer_id (FK), amount, timestamp
- **notifications:** id (PK), vendor_id (FK), title, message, type (INFO/WARNING/SUCCESS), is_read, timestamp

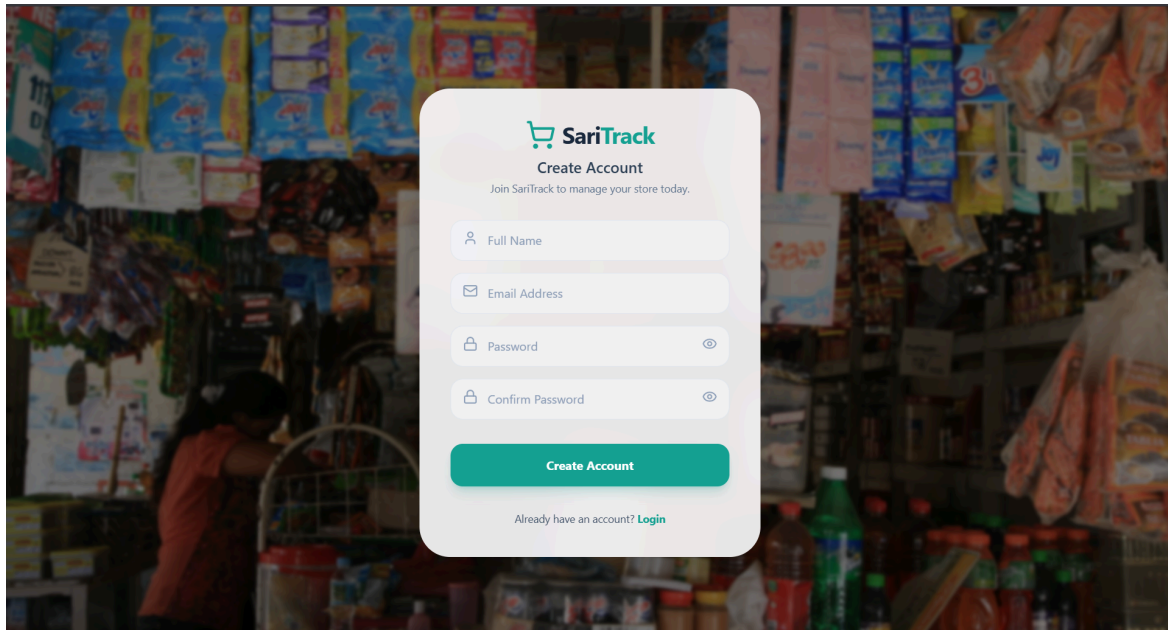
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Figma Link :

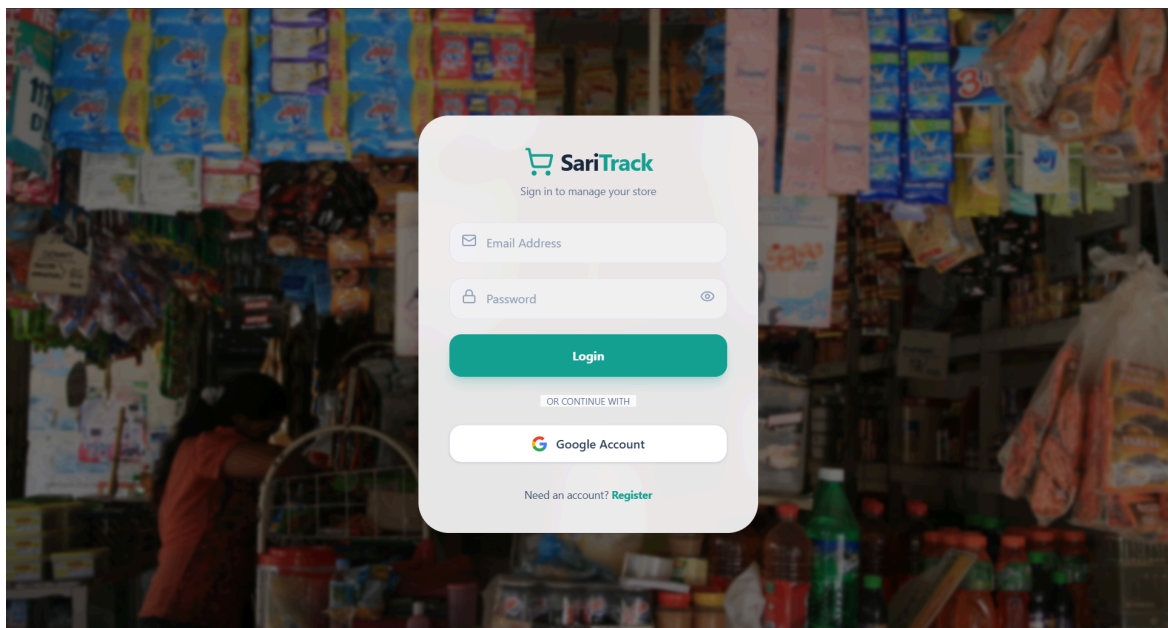
<https://www.figma.com/design/TBBRkHJtzId0oerTmDYapK/SariTrack?node-id=0-1&p=f&t=hJrdh6T94kOP5asA-0>

Sign Up



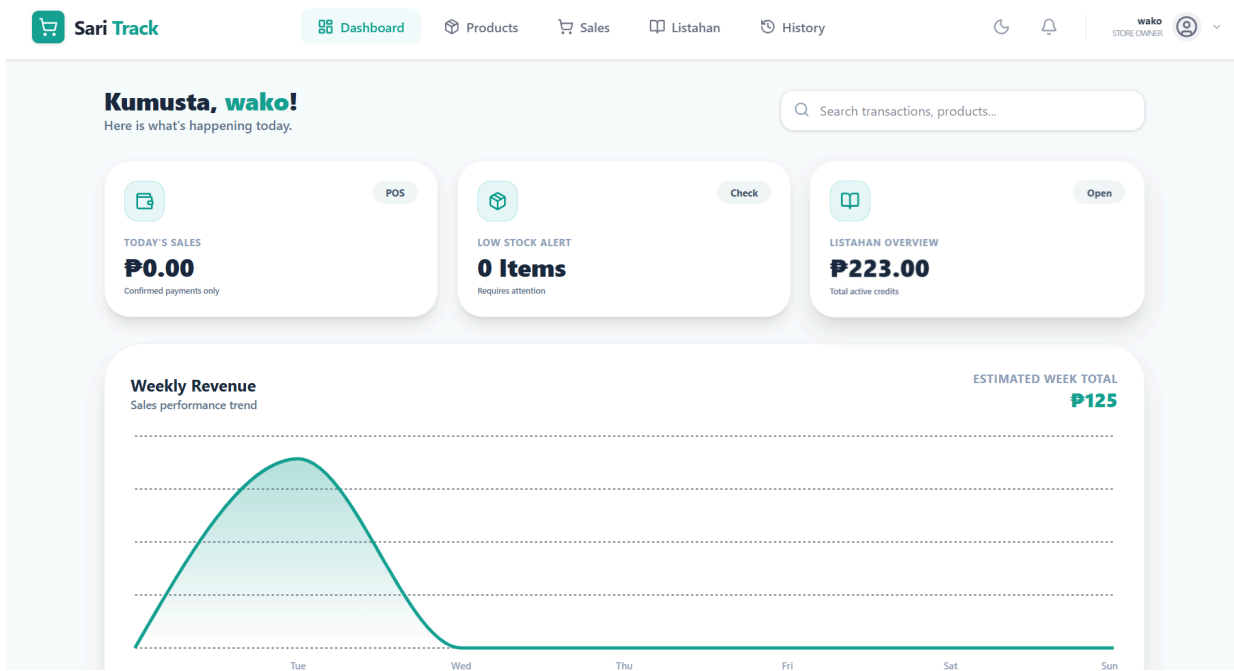
The Sign Up wireframe is overlaid on a background image of a small shop. It features the SariTrack logo at the top, followed by the heading 'Create Account' and a subtext 'Join SariTrack to manage your store today.' Below this are four input fields: 'Full Name', 'Email Address', 'Password' (with a toggle icon), and 'Confirm Password' (with a toggle icon). A green 'Create Account' button is positioned below the fields. At the bottom, there is a link 'Already have an account? Login'.

Sign In



The Sign In wireframe is overlaid on the same shop background. It features the SariTrack logo at the top, followed by the heading 'Sign in to manage your store'. Below this are two input fields: 'Email Address' and 'Password' (with a toggle icon). A green 'Login' button is positioned below the fields. Below the button is a link 'OR CONTINUE WITH' followed by a 'Google Account' button with the Google logo. At the bottom, there is a link 'Need an account? Register'.

Main Dashboard



Transactions :

Dashboard

Products

Sales

Listahan

History

wako

STORE OWNER

Transaction History

Review and manage all your store sales.

Export Report

Search Order ID...

ALLPAIDPENDINGCANCELLED

ORDER ID	DATE & TIME	AMOUNT	ITEMS	STATUS	ACTION
#80	5/19/2026 12:23 PM	₱125.00	1 items	PAID	Details
#79	5/14/2026 10:55 PM	₱46.00	1 items	PAID	Details
#78	5/14/2026 10:52 PM	₱50.00	1 items	DEBT	Details
#77	5/14/2026 10:52 PM	₱6.00	1 items	PAID	Details
#76	5/14/2026 10:45 PM	₱24.00	2 items	PAID	Details

Product Detail Page

DashboardProductsSalesListahanHistory

WakoStore Owner

Inventory

Manage your items and stock levels.

ExportAdd Product

Platos

N/A

PRICE

P1.00

STOCK

5

SafeGuard

N/A

PRICE

P39.00

STOCK

11

Yahoo

N/A

PRICE

P1.00

STOCK

77

Oishi

N/A

PRICE

P6.00

STOCK

16

Shades

N/A

Plastic king 3 X 16

4809011333079

Sardines

4806508627215

Wala

N/A

Admin Dashboard

SariAdmin

SYSTEM CONTROL

MANAGEMENT

DashboardVendorsGlobal Products

SYSTEM

Settings

PLATFORM CONTROL

Kyle Tapales

ROOT ADMINISTRATOR

Platform Control, Kyle!

Global system status and platform health.

MAINFRAME STABLE

LIFETIME PLATFORM SALES

P0.00

Total platform revenue (All-Time)

ACTIVE VENDORS

0 Stores

Current onboarded sellers

SYSTEM HEALTH

99.9%

API & Database Uptime

Global Platform Revenue

Aggregated sales trend across all nodes

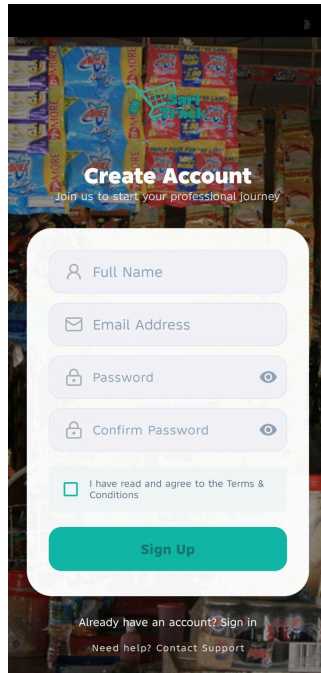
7-DAY PLATFORM TOTAL

P0

7.2 Mobile Application Wireframes

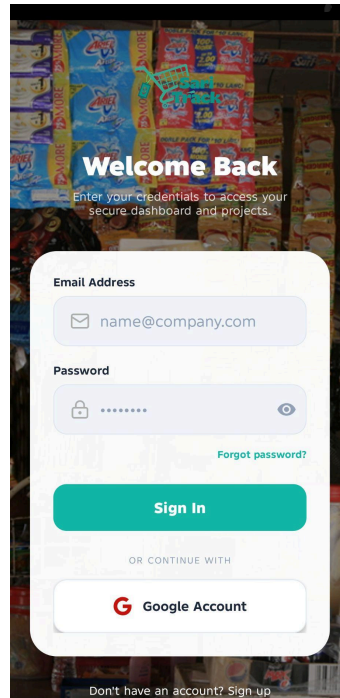
Note: This should be wireframes from Figma

Sign Up Screen



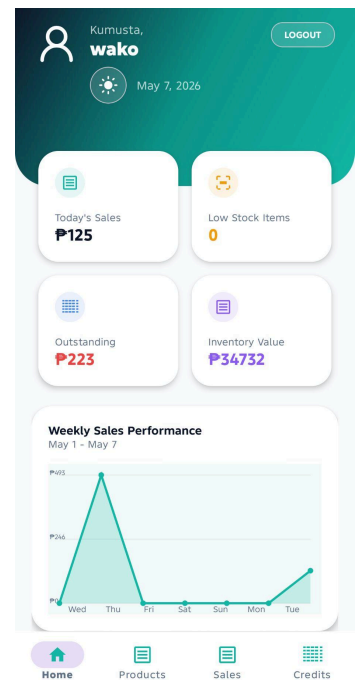
Wireframe for the Sign Up screen. It features a background image of a store shelf. The main heading is "Create Account" with a subtext "Join us to start your professional journey". Below this are four input fields: "Full Name", "Email Address", "Password" (with an eye icon), and "Confirm Password" (with an eye icon). There is a checkbox for "I have read and agree to the Terms & Conditions". A large teal "Sign Up" button is at the bottom. At the very bottom, there are links: "Already have an account? Sign In" and "Need help? Contact Support".

Sign In Screen

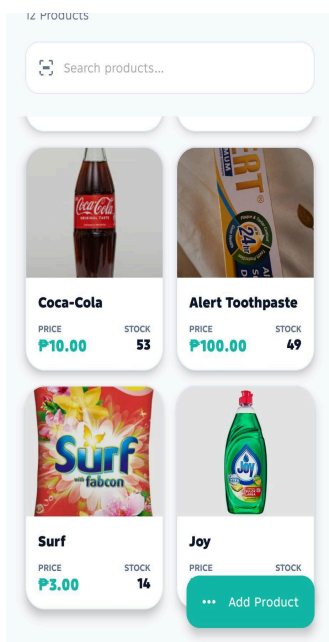


Wireframe for the Sign In screen. It features a background image of a store shelf. The main heading is "Welcome Back" with a subtext "Enter your credentials to access your secure dashboard and projects". Below this are three input fields: "Email Address" (with a pre-filled example "name@company.com"), "Password" (with a masked password "*****" and an eye icon), and a "Forgot password?" link. A large teal "Sign In" button is at the bottom. Below the button is a link "OR CONTINUE WITH" and a "Google Account" button with the Google logo. At the very bottom, there is a link "Don't have an account? Sign up".

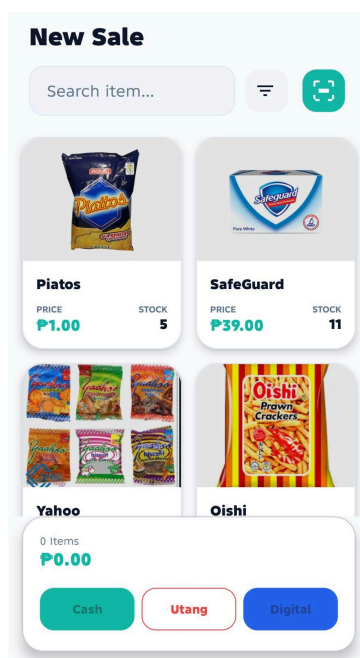
Main Dashboard



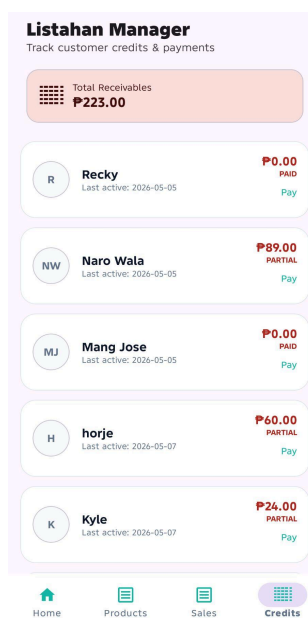
Product Screen



Sales Screen

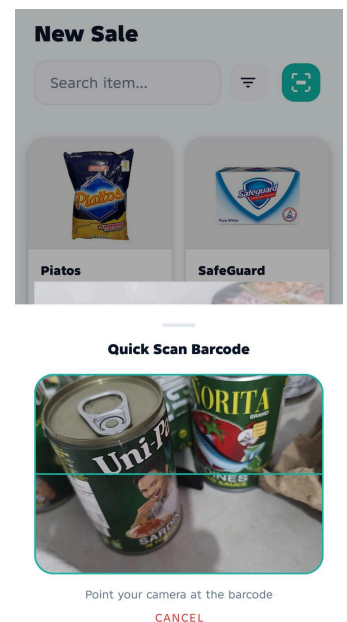


Listahan Screen



Wireframe for the Listahan Screen. It features a background image of a store shelf. The top section has a search bar labeled "Search item...". Below the search bar are four product cards: "Piatos" (PRICE P1.00, STOCK 5), "SafeGuard" (PRICE P39.00, STOCK 11), "Yahoo" (PRICE P0.00, STOCK 0), and "Oishi" (PRICE P0.00, STOCK 0). At the bottom is a button "Add Product".

Bar Code Scanner



Mobile-Specific Features:

- **Mobile Focus Architecture:** Employs a barcode-first point-of-sale workflow driven by Google's ML Kit Barcode Scanning API via custom camera layouts, providing instantaneous inventory lookups.
- **Navigation Architecture:** Implements a fixed material BottomNavigationView menu configuration (bottom_nav_menu.xml) to manage fluid tab transitions across core application fragments (HomeFragment, ProductsFragment, SalesFragment, and CreditsFragment).
- **Touch-Optimized Layout Targets:** interactive elements within all classical XML layouts enforce a strict minimum clickable touch boundary of **48x48dp** (density-independent pixels) matching Android Material Design accessibility standards, preventing misclicks in fast-paced retail environments.
- **Volatile Shopping Basket Utility:** Features an inline mobile CartBottomSheet and localized CartManager runtime array system, allowing vendors to dynamically adjust multi-item transaction quantities via touch-friendly increment components before checking out.
- **Optimized Input Sheets:** Leverages scrolling contextual layout views (activity_add_product.xml, layout_customer_picker.xml) paired with specialized floating sheets to minimize keyboard fragmentation and streamline one-handed retail data entry.

Design System:

Color Tokens (res/values/colors.xml)

The system enforces a high-contrast palette supporting accessible visual hierarchy across dark and light layout configurations:

- **Primary (Teal Accent):** #0D9488 (Maps core brand actions, button surfaces, and navigation highlights)
- **Secondary (Warning Orange):** #FB923C (Highlights credit visual elements and low-stock dashboard status counters)
- **Success (Emerald Green):** #10B981 (Represents paid status badges, successful scanner hits, and completed transactions)
- **Error (Crimson Red):** #EF4444 (Flags invalid barcode errors, stock depletion alerts, and validation issues)

- **Surface Background Layers:** Styled natively via surface tokens (bg_surface.xml, bg_dashboard.xml) using a neutral canvas palette.

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Weeks 1–2)

- **Week 1: Requirements & Architecture**
 - **Days 1–2:** Project environment initialization, scope definitions, and documentation setup.
 - **Days 3–4:** Verification of functional and non-functional multi-tenant SaaS requirements.
 - **Days 5–7:** Multi-tier architectural design layout and component flow mapping.
- **Week 2: As-Built Detailed Design Specification**
 - **Days 1–2:** REST API capsule layout definition and HTTP status mapping.
 - **Days 3–4:** Relational schema modeling and PostgreSQL multi-tenant boundary constraint design.
 - **Days 5–6:** Figma high-fidelity web dashboard and mobile interface wireframe composition.
 - **Day 7:** Baseline system architecture and implementation strategy finalization.

Phase 2: Backend Core API Development (Weeks 3–4)

- **Week 3: Architectural Foundation & Identity Security**
 - **Day 1:** Spring Boot 3.x setup with Spring Data JPA and PostgreSQL database initialization.
 - **Day 2:** RELATIONAL DATA LAYER configuration (Entities for User, Product, Customer, Order, OrderItem, Payment).
 - **Day 3:** Stateless JWT filter handling and Spring Security matrix configuration.
 - **Day 4:** Authentication controllers, user context lookup endpoints, and custom exception handler mapping.

- **Day 5:** Multi-tenant product inventory catalog management endpoints.
- **Week 4: Business Logic Domains & Automated Verification**
 - **Day 1:** Core order management tracking logic and transactional checkout service mapping.
 - **Day 2:** Multi-tenant customer ledger ("Listahan" tracking tracking) service and credit allocation endpoints.
 - **Day 3:** PayMongo Sandbox checkout session initialization and webhook response processing handlers.
 - **Day 4:** Live conversion rate lookup utility integration and system automated SMTP notification engine layout.
 - **Day 5:** Automated programmatic unit and regression testing implementation (BoundaryVerificationTest.java, SecurityMatrixTest.java).

Phase 3: Web Dashboard Application (Weeks 5–6)

- **Week 5: Frontend Foundation & Identity Lifecycle**
 - **Day 1:** React 18 single-page application structure and Tailwind CSS framework integration.
 - **Day 2:** Secure login, vendor onboarding forms, and localStorage context synchronization tracking.
 - **Day 3:** Responsive product inventory tables with multi-tenant filtering.
 - **Day 4:** Product entry sheet layout with dynamic payload validations.
 - **Day 5:** Multi-tenant customer credit tracking dashboard view.
- **Week 6: POS Cart Flow & Administrative View Matrix**
 - **Day 1:** Inline checkout workflow containing real-time input fields and debt selection items.
 - **Day 2:** Historic sales reporting boards and sandbox ledger checkout buttons.
 - **Day 3:** High-level platform administration portal for managing vendors and tracking site-wide usage analytics.
 - **Day 4:** Interface responsiveness tuning using modern CSS design tokens.

- **Day 5:** End-to-end Axios API client client endpoint linking and state-recovery optimization.

Phase 4: Native Android POS Mobile Application (Weeks 7–8)

- **Week 7: Mobile SDK Architecture & Scanning Interface**

- **Day 1:** Android Studio environment initialization, Gradle configurations, and material dependencies mapping.
- **Day 2:** Standard onboarding layouts and secure EncryptedSharedPreferences login token interceptor tracking.
- **Day 3:** Barcode scanning interface configuration using Google's camera-linked ML Kit Barcode Scanning API.
- **Day 4:** Volatile transaction management layer featuring localized CartManager utility memory arrays.
- **Day 5:** Network connection binding layer featuring Retrofit 2 API calls.

- **Week 8: Ledger Integrations & Mobile Interface Polish**

- **Day 1:** Bottom sheet cash checkout flow and atomic inventory transaction logging layout.
- **Day 2:** "Listahan" ledger tracking module and dynamic customer selection overlays (CustomerPickerBottomSheet).
- **Day 3:** Retrofit endpoint alignment alongside local Room DB offline lookup cache optimization.
- **Day 4:** Interface enhancement involving high-contrast color codes and standard touch-optimized Material boundaries (**48x48dp** targets).
- **Day 5:** Production compiler execution, debugging, and distribution bundle compilation.

Phase 5: Integration, Verification & Deployment (Weeks 9–10)

- **Week 9: System-Wide Validation Testing**

- **Day 1:** Full-stack integration tracking verifying web dashboard actions against central backend endpoints.
- **Day 2:** Cross-platform validation tracking ensuring mobile camera barcode scans sync with database tables.

- **Day 3:** Webhook processing checks using the PayMongo Sandbox dashboard interface.
- **Day 4:** Data isolation verification testing to confirm individual vendor accounts are completely locked away from other store owners.
- **Day 5:** Code optimization, performance adjustments, and documentation assembly.
- **Week 10: Production Release Operations**
 - **Day 1:** Spring Boot API deployment to hosted cloud platforms (Railway/Render).
 - **Day 2:** React single-page frontend deployment to secure production endpoints (Vercel/Netlify).
 - **Day 3:** Distributed Android mobile APK packaging and local database installation checks.
 - **Day 4:** Live multi-platform end-to-end user experience checks.
 - **Day 5:** System deployment validation finalization and academic portfolio file compilation.

8.2 Project Milestones

- **M1 (End of Week 2):** High-fidelity architectural maps, schema documentation, and API contracts finalized.
- **M2 (End of Week 4):** Secure backend API containing automated test coverage fully operational on a database.
- **M3 (End of Week 6):** React Web Management Dashboard application complete and connected to backend data routers.
- **M4 (End of Week 8):** Native Kotlin Android mobile scanning client fully operational across checkout views.
- **M5 (End of Week 10):** Integrated cloud ecosystem running flawlessly across production endpoints.

8.3 Critical Path

1. **Authentication & Cryptographic Tokens (Week 3):** The cornerstone security configuration that locks down all protected data paths across the entire multi-tenant system.

2. **Product Catalog & Multi-Tenant Data Structures (Weeks 3–4):** Core entity models required for inventory lookup queries and scanning integrations.
3. **Atomic Sales Transactions & Debt Ledgers (Weeks 4, 6, 8):** Key business processing blocks handling inventory count tracking and debtor entry logging.
4. **Cross-Platform Integration (Week 9):** Unified testing validating client applications against real-time production backend routes.

8.4 Risk Mitigation Strategy

- **Volatile Memory Optimization:** Use localized data collections (CartManager runtime arrays) to build high-performance mobile checkout tools without adding extra database table overhead.
- **Continuous Multi-Platform Validation:** Run end-to-end integration checks early and often to catch and resolve structural changes or mismatched types before building final deployment bundles.
- **Programmatic Security Verification:** Use automated boundary tests to verify that data rows stay completely separated between vendors on every build.
- **Isolate Experimental Features:** Perfect the essential POS scanner flows and customer ledger tracking blocks before adding cosmetic animations or secondary user features.